ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Доцент, канд. тех. наук		Степанов П. А.
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Разработка формы логина

по курсу: Технология разработки серверных информационных систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4434		Энгельгардт К. А.
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1. Текст и вариант задания

Вариант задания: 1 - Книжный магазин либо библиотека.

Задание на лабораторную работу:

- Используйте в качестве основы лабораторную работу номер 3
- Переведите вебсервер на использование SSL
- Реализуйте form-based authentication
- Защитите ресурсы, требующие запись и введите аудит.
- Форма должна содержать кнопку Logout.

2. Описание разрабатываемого продукта

Разрабатываемый программный продукт представляет собой защищённое клиент-серверное веб-приложение для управления каталогом книжного магазина:

- Архитектура: Spring Boot 3.2.5 (серверная часть на Java 17) + Thymeleaf + AngularJS 1.7.5 (клиентская часть).
- Безопасность сетевого уровня (SSL/TLS): Приложение функционирует исключительно по защищённому протоколу HTTPS на порту 8443 с использованием сертификата стандарта PKCS12 (keystore.p12).
- Аутентификация (Form-Based Authentication): Доступ к системе осуществляется через кастомную HTML-форму логина (/login) со стилизацией, описанной в методичке (main-div, panel, form-heading). Пароли пользователей защищены алгоритмом хеширования BCrypt.
- Разграничение доступа (Role-Based Access Control):
- Пользователь с ролью USER (user / password): имеет право только на просмотр каталога (HTTP GET).
- Пользователь с ролью ADMIN (admin / admin): имеет полный доступ на создание, редактирование и удаление книг (HTTP POST, PUT, DELETE).
- Аудит действий: Все изменяющие операции в каталоге фиксируются выделенным сервисом аудита (AuditService) с записью в системный журнал имени текущего пользователя, совершившего операцию, типа действия и идентификатора записи.
- Защита от межсайтовой подделки запросов (CSRF): Включена поддержка CSRF-токенов в формах и настроена автоматическая передача токена в AJAX-запросах AngularJS через заголовок X-CSRF-TOKEN.
- Завершение сеанса (Logout): На главной странице расположена кнопка выхода, выполняющая защищённый POST-запрос на /logout с инвалидацией пользовательской сессии и перенаправлением на страницу логина.

3. Текст основных фрагментов кода

3.1. Конфигурация SSL в application.properties

```
server.port=8443

spring.application.name=bookstore

# SSL / HTTPS (методичка стр. 47)

server.ssl.key-store=classpath:keystore.p12

server.ssl.key-store-password=mypassword

server.ssl.keyStoreType=PKCS12

server.ssl.keyAlias=tomcat

# Логирование аудита

logging.level.ru.spbguap.bookstore=DEBUG

logging.level.org.springframework.security=INFO
```

3.2. Конфигурация безопасности SecurityConfig.java

```
@Configuration

@EnableWebSecurity

public class SecurityConfig {



    @Bean

    public SecurityFilterChain filterChain(HttpSecurity http)
throws Exception {

        http

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/login", "/css/**",
"/js/**").permitAll()

                .requestMatchers(HttpMethod.GET, "/api/books",
"/api/books/**").authenticated()

                .requestMatchers(HttpMethod.POST,
"/api/books/**").hasRole("ADMIN")

                .requestMatchers(HttpMethod.PUT,
"/api/books/**").hasRole("ADMIN")
```

```
                .requestMatchers(HttpMethod.DELETE,
"/api/books/**").hasRole("ADMIN")
                .requestMatchers("/swagger-ui/**", "/api-
docs/**").authenticated()
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .usernameParameter("login")
                .passwordParameter("pass")
                .permitAll()
                .defaultSuccessUrl("/books", true)
            )
            .logout(logout -> logout
                .logoutRequestMatcher(new
AntPathRequestMatcher("/logout"))
                .logoutSuccessUrl("/login?logout")
                .permitAll()
            )
            .csrf(csrf -> csrf
                .ignoringRequestMatchers("/api/**")
            );

        return http.build();
    }

    @Bean
```

```
    public UserDetailsService userDetailsService(PasswordEncoder
passwordEncoder) {
        var user = User.builder()
                .username("user")
                .password(passwordEncoder.encode("password"))
                .roles("USER")
                .build();

        var admin = User.builder()
                .username("admin")
                .password(passwordEncoder.encode("admin"))
                .roles("ADMIN", "USER")
                .build();

        return new InMemoryUserDetailsManager(user, admin);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

3.3. Сервис аудита AuditService.java и аудит в BookService.java

```
@Service
public class AuditService {
    private static final Logger auditLog =
LoggerFactory.getLogger("AUDIT");
```

```
    private String currentUser() {
        Authentication auth =
SecurityContextHolder.getContext().getAuthentication();
        return (auth != null && auth.isAuthenticated()) ?
auth.getName() : "anonymous";
    }

    public void logCreate(String entity, Object id) {
        auditLog.info("[AUDIT] user='{}' action=CREATE entity={}
id={}", currentUser(), entity, id);
    }

    public void logUpdate(String entity, Object id) {
        auditLog.info("[AUDIT] user='{}' action=UPDATE entity={}
id={}", currentUser(), entity, id);
    }

    public void logDelete(String entity, Object id, boolean
success) {
        auditLog.info("[AUDIT] user='{}' action=DELETE entity={}
id={} success={}", currentUser(), entity, id, success);
    }
}
```

3.4. Шаблон формы логина login.html

```
<div class="main-div">
    <div class="panel">
        <h2 class="form-heading">Our University Database</h2>
        <p>Please enter your login and password</p>
```

```
        <div th:if="${param.error}" class="alert alert-
danger">Unknown username or password.</div>
        <div th:if="${param.logout}" class="alert alert-
success">Good bye!</div>
    </div>

    <form id="Login" th:action="@{/login}" method="post">
        <div class="form-group">
            <label for="login">Логин:</label>
            <input type="text" class="form-control" id="login"
name="login" placeholder="Login" required autofocus/>
        </div>

        <div class="form-group">
            <label for="pass">Пароль:</label>
            <input type="password" class="form-control" id="pass"
name="pass" placeholder="Password" required/>
        </div>

        <button type="submit" class="btn btn-
primary">Login</button>
    </form>
</div>
```

3.5. Форма Logout и настройка CSRF в AngularJS (books.html)

```
<div class="user-panel">
    <span sec:authorize="isAuthenticated()">
        Вы вошли как: <b><span
sec:authentication="name">User</span></b>
```

```
        (роль: <span
sec:authentication="principal.authorities">ROLE</span>)

    </span>

    <form name="logoutForm" th:action="@{/logout}" method="post"
style="display: inline; margin-left: 10px;">

        <input type="submit" class="btn-logout" value="Sign Out"/>

    </form>

</div>

// Настройка AngularJS с передачей CSRF-токена в AJAX-запросы

var app = angular.module('bookstoreApp', []).config(function
($httpProvider) {

    var csrftoken = $("meta[name='_csrf']").attr("content");

    var csrfheader =
$("meta[name='_csrf_header']").attr("content");

    if (csrftoken && csrfheader) {

        $httpProvider.defaults.headers.common["X-CSRF-TOKEN"] =
csrftoken;

    }

});
```

Заключение

В ходе выполнения лабораторной работы №4 веб-приложение «Книжный магазин» было успешно переведено на защищённый протокол HTTPS с использованием SSL-сертификата стандарта PKCS12 на порту 8443. Реализована аутентификация пользователей на основе веб-формы (Form-Based Authentication) с защитой паролей стойким алгоритмом хеширования BCrypt и обработкой состояний входа и выхода. Настроено ролевое разграничение доступа (RBAC), при котором операции чтения доступны всем авторизованным пользователям, а действия, требующие записи (создание, редактирование и удаление книг), ограничены ролью администратора и сопровождаются обязательным аудитом с фиксацией имени пользователя в системном журнале. Для завершения сеанса реализована кнопка Logout с защищённой отправкой POST-запроса и инвалидацией сессии, а также настроена передача CSRF-токена в AJAX-запросах библиотеки AngularJS, что обеспечивает комплексную защиту приложения от типовых сетевых угроз.